

Abstract

Job seekers and recruiters today interact with recommendation systems that produce ranked lists but rarely explain themselves, leaving users unable to judge whether a top match is worth their attention or how a profile change would move their ranking. We introduce JOBMATCH, an interactive, explainable agentic system that supports transparent career decision-making by combining role-aware document analysis, hybrid semantic-skill matching, and structured natural-language explanations with a calibrated confidence display. Grounded in four design principles derived from prior human-AI interaction research—transparency, explainability, user control, and multi-perspective decision support—the system organizes the matching workflow as three cooperating interaction components: a candidate-side component that helps users understand and revise what the system knows about them, an employer-side component that helps recruiters understand which requirements drive a ranking, and a read-only matchmaking component that returns ranked lists with component-level reasons. We evaluated the system on a 30-resume, 15-job demo corpus with 47 manually labeled pairs. Hybrid ranking combining resume text and skills reached $nDCG@5 = 0.969$ and a portal-default composite reached 0.949, with explanations showing 0.745 faithfulness and confidence scores calibrated to an expected calibration error of 0.032. The contribution is a design artifact and an evaluation methodology for interactive, explainable agentic job recommendation, not a deployment study: every consequential outcome—apply, shortlist, contact—remains an explicit user action.

Keywords: Agentic AI, Human-AI Interaction, Explainable AI, Trust Calibration, User Control, Job Recommendation, Interactive Recommender Systems, Career Decision Support, LLM Agents.

Understanding, Controlling, and Trusting Agentic AI: An Interactive, Explainable Job-Matching System

Anonymous Authors
Anonymous Institution, Anytown, AnyState, AnyCountry
anonymous@example.com *

August 1, 2026

1 Introduction

Online hiring platforms present job seekers and recruiters with ranked lists, but the reasons behind a given ranking are typically hidden behind a single opaque score. A candidate who edits a resume to add a new skill has no way to see whether the new skill moved a posting up or down the list, or whether the change reached the system at all. A recruiter who changes a job description and reruns a search receives a refreshed shortlist, but the link between the edit and the resulting rank order is invisible. Both sides of the market therefore share a common problem: an absence of feedback that would let them reason about, contest, or learn from the recommendations they receive. This paper asks what it would take to design a career-recommendation system whose interaction surface makes its reasoning legible to the people whose decisions it is meant to support.

A growing body of work on agentic systems based on large language models (LLMs) makes this question timely. LLM-based agents can parse resumes into structured fields, normalize skills against a shared vocabulary, draft natural-language explanations of ranking decisions, and chain such subtasks together ??????. Multi-agent designs further distribute the work: a candidate-facing agent can maintain a versioned user model while a separate employer-facing agent maintains a job model, and a third, neutral component can compare the two without ever writing into either store ??????. This delegation of labor is, however, only the first step. What such systems do for the person reading a recommendation is a separate question, and a more difficult one. A model that scores well offline can still leave users worse off if its explanations mislead, if its confidence is poorly calibrated, or if it removes the user’s ability to act.

Three concrete challenges motivate our work. First, existing recommender systems are largely opaque from the user’s perspective: the dominant paradigm in production hiring tools returns a list and a single number, with no breakdown of which input fields contributed ???. Second, the explanations that have been studied in the literature are often post-hoc reconstructions of model behavior, not direct accounts of the inputs that drove a score; users have repeatedly been shown to over- or under-rely on such accounts depending on their framing ????. Third, the arrival of LLM-based agents in recommendation pipelines introduces new surface area for trust and control problems: a system that can both rank and explain can also rank incorrectly while sounding confident ?.

*During preparation of this work, the author(s) used a large language model (GPT-4 class) to assist with copy-editing of selected paragraphs and to produce initial drafts of figure captions from a structured outline. All technical content, experimental results, and design decisions are the authors’ own. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

We address these challenges with JOBMATCH, a research prototype that organizes a job-candidate matching workflow as three cooperating interaction components rather than as a single end-to-end predictor. The candidate-side component helps users inspect, revise, and understand what the system knows about them, and emits an event only after the user confirms a save. The employer-side component plays the symmetric role for job descriptions, exposing which requirements are driving a shortlist and which are inert. A read-only matchmaking component consumes the resulting snapshots, returns a ranked list, attaches component-level reasons, and reports a calibrated confidence value. The user is the locus of decision in all three components: applying, shortlisting, contacting, and dismissing remain explicit human actions that the system records but does not act on.

Contributions

This paper makes the following contributions. *First*, we contribute a set of design principles for interactive, explainable agentic career recommendation, derived from established guidelines for human–AI interaction [1, 2], and instantiate them as concrete interaction patterns in a working system. *Second*, we present a multi-component agentic architecture in which each component is described by the user need it addresses, the information it returns, and the control it preserves, rather than by its internal data flow. *Third*, we report a controlled evaluation on a fixed demo corpus that simultaneously measures ranking quality (nDCG@5), explanation faithfulness, confidence calibration (ECE), and adversarial robustness under controlled profile edits, and we map each metric to a user-facing property. *Fourth*, we provide an anonymized open-source artifact that includes the prototype, the demo corpus, the explanation generator, the calibration layer, and the regression benchmark, so that the design choices and their measured effects can be inspected and replicated.

The remainder of the paper is organized as follows. Section 2 situates the work in the literature on interactive recommender systems, explainable AI, human–AI decision making, and LLM-based agents. Section 3 derives the design principles that guide the prototype. Section 4 describes the system in terms of user needs, component responsibilities, and the boundaries between them. Section 5 presents the interactive interface as a sequence of design choices visible in the running prototype. Section 6 defines the evaluation methodology and maps each metric to a user-facing property. Section 7 reports results. Section 8 discusses implications, limitations, and future work. Section 9 concludes.

2 Related Work

The work reported here sits at the intersection of four research streams: interactive and explainable recommender systems, human–AI decision support, LLM-based agentic systems, and the specific literature on algorithmic hiring. We briefly review each stream and identify the gap that motivates our design.

Interactive and explainable recommender systems

Modern recommender systems rely on learned dense representations, often combined with sparse lexical signals, to produce personalized rankings [3]. Tintarev and Masthoff [4] catalog the design space for recommender explanations and report that explanations are most useful when they address a user’s specific question rather than narrating the model’s general behavior. Pu and Chen [5] show that explanation interfaces can raise user trust in recommendation agents when the explanation style matches the user’s task. Ziegler and colleagues [6] demonstrate that diversification of recommendation lists can reduce over-reliance on a single signal. More recent work has examined the fairness implications of recommender pipelines and the need to surface accountability at the recommendation surface [7]. Our system inherits the recommendation backbone of this literature but adds an interactive layer in which the user can see, before ranking, what the system has parsed and what it will compare.

Explainable AI

A substantial body of work in XAI argues that explanations should be evaluated for what they enable users to do, not for their fidelity to an internal model ???. Liao and colleagues ? review XAI user-experience practice and report that the most common failure mode is explanation content that is internally consistent but does not answer the user’s question. Ehsan and colleagues ?? propose that explanations should be evaluated against the social context in which the system is used, not against a model-internal notion of correctness. Joshi and colleagues ? reframe explainability as a property of the human–AI decision process rather than of the model in isolation. We adopt this orientation: the explanations in our prototype are evaluated for their faithfulness to the components that produced them and for their usefulness to the user, not for their match to any single internal representation.

Human–AI decision support and trust calibration

A long tradition in human factors research treats trust in automation as something that must be calibrated to system capability rather than encouraged or discouraged wholesale ???. In machine-learning systems, this has produced a literature on confidence calibration, including the use of Platt scaling to recover well-calibrated probabilities from modern neural networks ??, the expected calibration error as a standard reporting metric, and the Brier score as a complementary measure ?. Mixed-initiative systems, in which the human and the system negotiate the locus of action, provide the conceptual ancestor of the design choices in our prototype ????. Guidelines for human–AI interaction consolidate these findings into actionable design practice ????. We draw directly on this body of work to derive the design principles in Section 3.

LLM-based agentic systems

LLM-based agents decompose complex tasks into tool calls, intermediate reasoning steps, and communication among specialist components ?????. In collaborative-agent designs, separate LLM-driven components take on distinct roles and exchange messages or artifacts ????. Cognitive architectures for language agents argue that a separation between reasoning, memory, and action is necessary for agents that operate over long horizons ?. In the multi-agent systems tradition, the role-separation idea is older and well developed ??. Our work is influenced by both traditions but is positioned differently: the agents in our system are not the contribution; the user-facing interaction patterns that the agents make possible are.

Algorithmic hiring and decision support in the recruitment domain

Hiring is a domain in which opacity and trust failures have well-documented consequences ??. A line of recent work in recommender systems targets fairness and accountability specifically in the recruitment setting ?. At the same time, the HCI community has repeatedly shown that decision support in high-stakes domains must preserve user agency and must not collapse the user into a passive recipient of model output ????. Our prototype is designed against this constraint: every consequential outcome in the user interface requires an explicit action and remains reversible.

Positioning

The present work is, to our knowledge, the first to combine four properties in a single, end-to-end research prototype for career recommendation: (i) an agentic, role-separated architecture in which no component writes into another component’s store; (ii) explanations that are anchored to the components that produced them and are evaluated for faithfulness; (iii) a calibrated confidence display that exposes the system’s uncertainty to the user; and (iv) a working interactive interface in which user actions on explanations feed back into the system rather than only consuming its output. The closest prior systems address one or two of these properties but not all four; we return to this comparison in Section 8.

3 Design Principles

The prototype described in Section 4 and the interface described in Section 5 instantiate four design principles that we derive here from prior work in human–AI interaction, explainability, and trust calibration. The principles are not claims about the system’s behavior; they are commitments against which the system’s design is judged. Section 7 returns to each principle and reports the empirical property that operationalizes it.

Principle G1: Transparency of state

A user must be able to see, before any recommendation is computed, what the system has parsed from their document and what it will compare. The principle is grounded in the observation that explanations of opaque model output are difficult to evaluate when the user cannot see the inputs to the model ?? . In our system, the candidate portal exposes the parsed fields of a resume as a review form before any snapshot is registered; the employer portal exposes the parsed requirements of a job description as a similar review form. A user who disagrees with a parsed field can edit it; the system does not rank on the basis of half-edited forms. This is the transparency property the system commits to: the data on which a ranking is computed is the same data the user last confirmed.

Principle G2: Faithful, component-level explanations

When the system returns a ranked list, every list item should carry an explanation that names the input fields that contributed to its position. The principle follows the XAI literature that ties explanation quality to the question a user is actually asking ?? and to recent work that warns against explanations that are internally consistent but decoupled from the components that produced the score ?. Our explanations are bound to the same composite channels that produced the score; we evaluate their faithfulness against the contributions of individual channels in Section 7.

Principle G3: User control at consequential points

Three actions in the system are consequential for the user: applying to a job, shortlisting a candidate, and contacting a candidate. None of them happens implicitly. Each requires a user gesture; the system records the action and updates local state but does not act on the user’s behalf. This commitment is the user-control counterpart of the trust-calibration literature ??? and of the mixed-initiative tradition that assigns the system a supporting role ??? . A useful interaction is one in which the user can always inspect what the system did, undo it if it was wrong, and decide what to do next.

Principle G4: Multi-perspective decision support

A ranking is rarely a single number in the user’s mind; it is a composite of independent considerations—text similarity, skill overlap, experience tier, compensation fit, and remote policy in our case. The system exposes these as separable components of the score rather than collapsing them into a single value, so that a user who weights remote policy more heavily than experience tier can read both components in the explanation panel. The principle is grounded in the recommendation literature that warns against over-reliance on a single signal ?? and in the interactive-machine-learning tradition that exposes intermediate signals to the user for inspection and revision ?? .

How the principles relate

The four principles are not independent. G1 supports G2: a user who can see what the system has parsed can evaluate an explanation that names those fields. G2 supports G3: a user who sees a component-level reason for a ranking is in a position to act on the components they disagree with. G3 supports G4: when users retain control over the final action, exposing the components of a score is informative rather than threatening. We refer back to this dependency in Section 8 when we discuss the design space our system does not address.

4 System Design

We describe JOBMATCH by the user needs it addresses, the components that address them, and the boundaries between them. The system is organized as three cooperating interaction components, each with a single user constituency and a clearly defined interface to the others. The data structures and algorithms are documented in the supplementary material; the focus here is the division of responsibility from the user’s point of view.

Overview

C1: Candidate-side component.

The candidate-side component maintains a structured representation of what the job seeker has told the system about themselves. It accepts a resume, parses it into discrete fields, normalizes the field values against a shared vocabulary, and presents the parsed result to the user as a review form. Only after the user confirms a save does the component register a versioned snapshot and emit a snapshot event to the matchmaking component. The component does not see job descriptions, does not call the matchmaking component directly, and does not act on the user’s behalf. Figure 2 shows the ingestion and representation pipeline; the user-facing surface is the review form, and the back-end store is the in-memory profile registry that backs the snapshot.

C2: Employer-side component.

The employer-side component plays the symmetric role for hiring organizations. It accepts a job description, parses it into required and preferred skills, experience bounds, compensation band, remote policy, and descriptive text, and presents the parsed result to a recruiter as a review form. Only on confirmation does it register a versioned snapshot. The component does not see candidate profiles and does not write to the matchmaking component’s store. Figure 3 shows the symmetric ingestion and representation pipeline; the user-facing surface is the recruiter review form, and the back-end store is the in-memory job registry.

C3: Matchmaking component.

The matchmaking component reads candidate and job snapshots by identifier, scores each feasible pair using a configured strategy, attaches component-level reasons, attaches a calibrated confidence value, and returns the ranked list. It never writes into a candidate or job store; it never calls C1 or C2 directly; it never acts on behalf of either side. When a snapshot event arrives, it drops cached rankings that depend on the changed entity and recomputes on the next request. Figure 4 shows the end-to-end matchmaking workflow: ingestion, snapshot, ranking, optional coach feedback, and user actions.

This three-component layout is, in the language of the prior submission, a multi-agent system; the difference in the present framing is the description of each component by the user need it addresses, the information it returns, and the control it preserves, rather than by its internal data flow. Figure 1 shows the high-level layout; Figure 5 traces the end-to-end flows for both sides.

What each component provides to the user

C1 (candidate-side). *User need:* a job seeker who has edited a resume should be able to verify what the system now knows about them and to revise it before the next recommendation. *Information provided:* parsed fields, normalized skills, a confidence flag for each field, the version of the snapshot currently in use, and a side-by-side view of the editable profile and the snapshot. *Control preserved:* the snapshot is updated only on user confirmation; the user can edit any parsed field; the user can revert to a previous snapshot. *Contribution to explanation:* the candidate-side component is the source of every field the matchmaking component names in an explanation, so a

High-Level Architecture of JobMatch

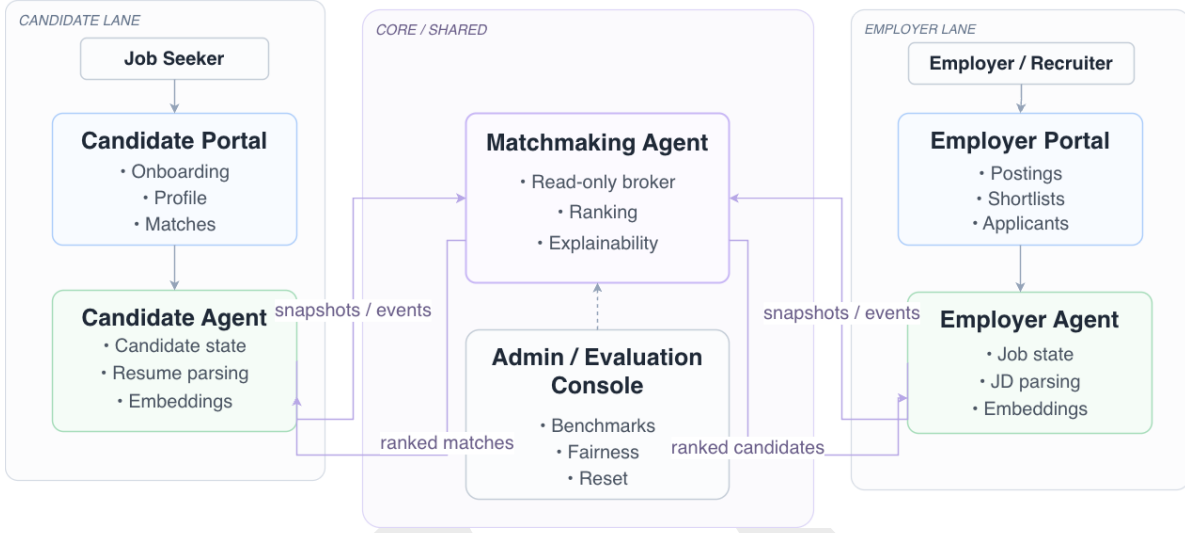


Figure 1: High-level layout of JOBMATCH. The candidate-side component (C1) and the employer-side component (C2) own the user-editable state on each side of the market. The matchmaking component (C3) reads those snapshots, returns a ranked list with component-level reasons and a calibrated confidence, and never writes into either store. The dashed arrows represent snapshot and event flow; the solid arrows represent user-facing recommendations and feedback.

candidate who reads a low score on a remote-policy component can locate the remote-policy field in the snapshot and revise it.

C2 (employer-side). User need: a recruiter who has revised a job description should be able to verify which requirements the system now considers and to revise them before the next shortlist. *Information provided:* required and preferred skills, experience bounds, compensation band, remote policy, descriptive text, the version of the snapshot currently in use, and a flag for each requirement that indicates whether it currently constrains the ranking. *Control preserved:* the snapshot is updated only on user confirmation; the user can toggle a requirement between required and preferred; the user can revert to a previous snapshot. *Contribution to explanation:* the employer-side component is the source of every requirement the matchmaking component names in a shortlist explanation.

C3 (matchmaking). User need: a user (candidate or recruiter) who has just confirmed a snapshot should be able to read, in the same interaction, what the system ranked and why, with enough detail to decide whether to act. *Information provided:* a ranked list, a component-level reason for each list item, a calibrated confidence value, a small set of counterfactual edits that would move the item, and an action menu that exposes apply, save, dismiss, and (on the employer side) shortlist and contact. *Control preserved:* the matchmaking component never acts on the user’s behalf; the user can dismiss an item, ask the component to re-rank with a constraint, or close the interaction without further action. *Contribution to explanation:* the matchmaking component is the only component that aggregates the snapshot channels, so it is the natural place to expose the per-channel contributions and the calibrated confidence.

Communication and state sharing

The three components exchange two kinds of artifacts. *Snapshots* cross the boundary from C1 and C2 to C3 as read-only data; the matchmaking component never sees the editable profile behind a snapshot, and C1 and C2 never see each other’s state. *Events* flow back from C1 and C2 to C3 when a snapshot is confirmed or reverted; they carry no editable content, only an identifier and a

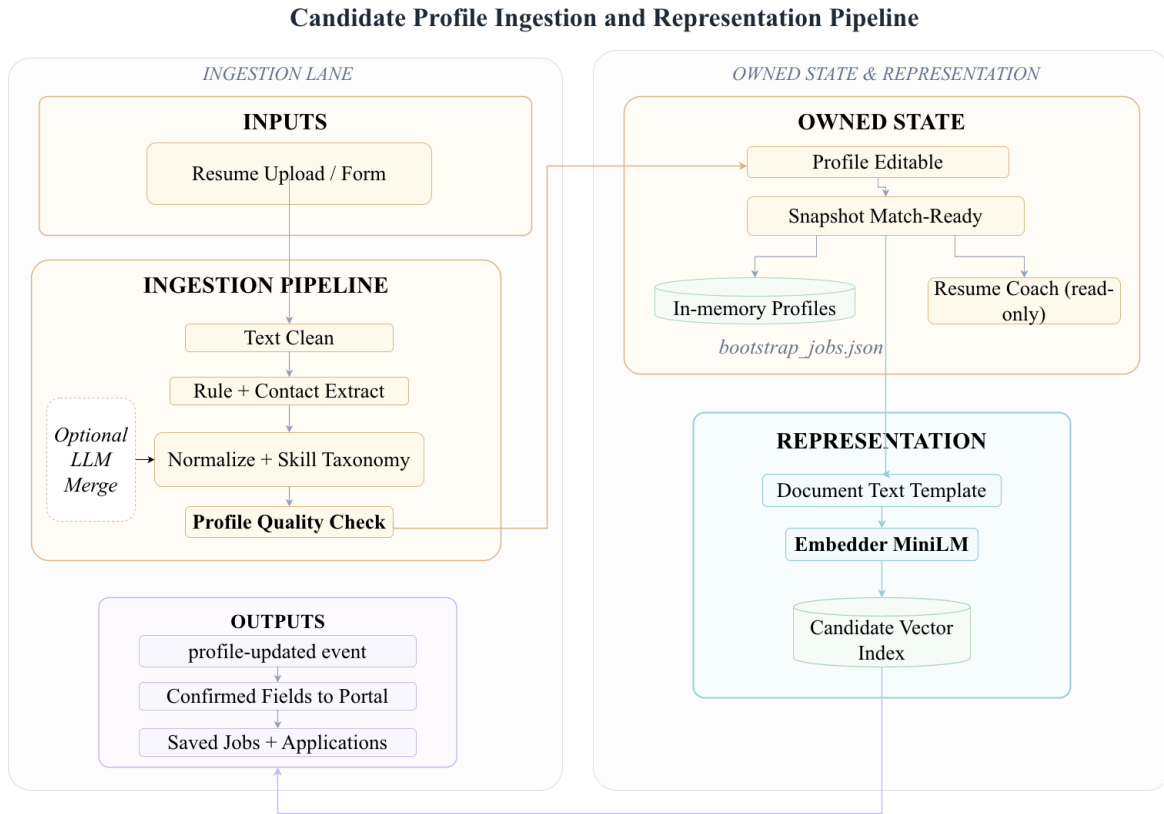


Figure 2: Candidate-side ingestion and representation pipeline. The candidate-side component owns the editable profile and the versioned snapshot. The ingestion lane accepts a resume upload, runs text clean and rule-based extraction, normalizes the result against the shared skill vocabulary (with an optional LLM fallback for ambiguous fields), and presents the parsed result to the user as a review form. Only on user confirmation does the component register a snapshot and emit a profile-updated event.

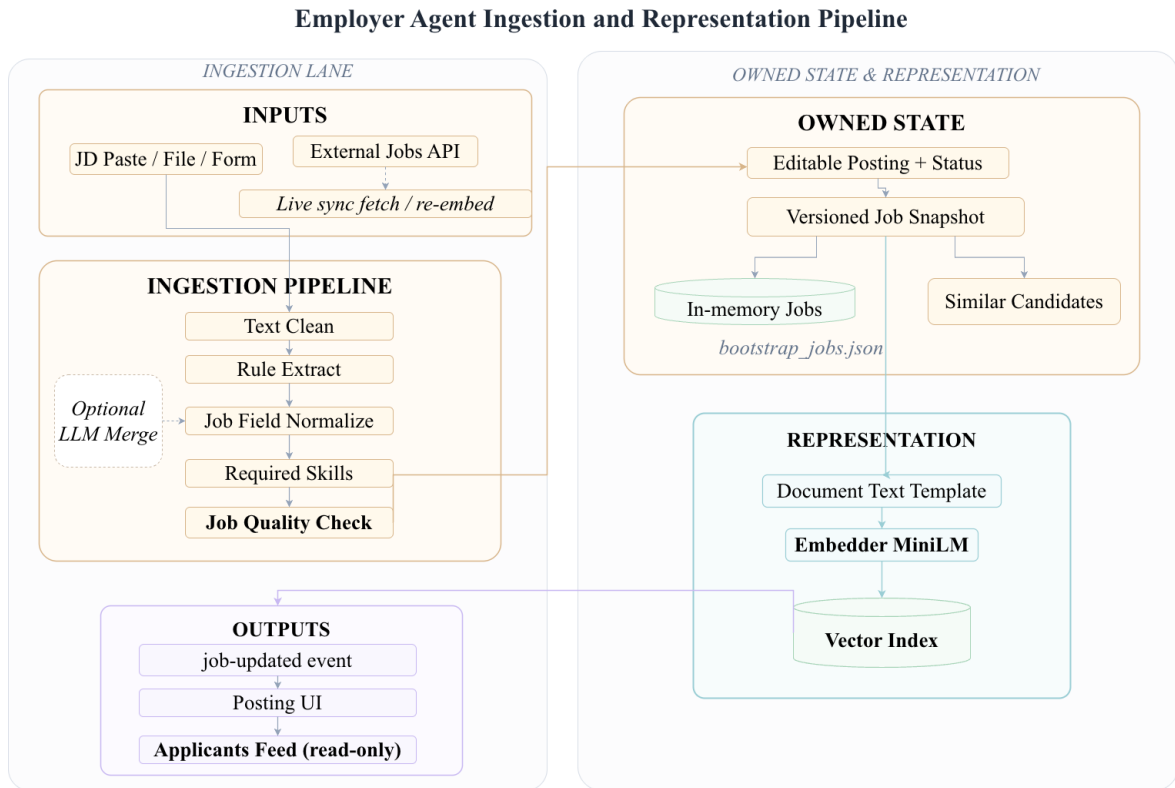


Figure 3: Employer-side ingestion and representation pipeline. The employer-side component mirrors the candidate-side layout for the job side: editable posting, versioned snapshot, and an ingestion lane that accepts a job description paste, file, or external API and presents the parsed result as a recruiter review form. Only on confirmation does the component register a snapshot and emit a job-updated event.

Employer Lifecycle and Matchmaking Workflow

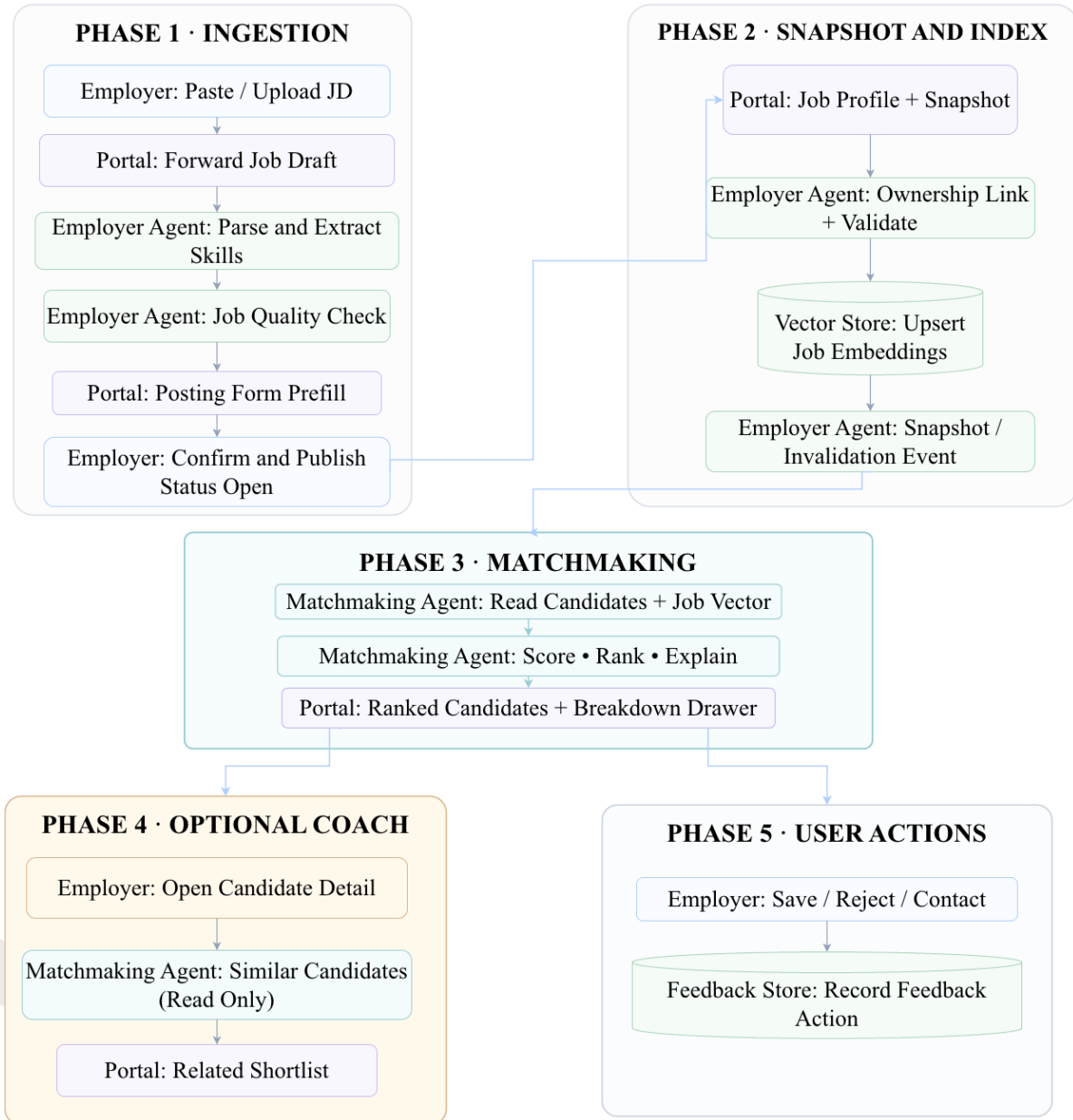


Figure 4: End-to-end matchmaking workflow. Ingestion, snapshot/index, ranking, optional coach feedback, and user actions are sequenced so the read-only matchmaking component never crosses the ownership boundary. The workflow is the same whether it is initiated by a candidate refreshing their matches or by an employer refreshing their shortlist.

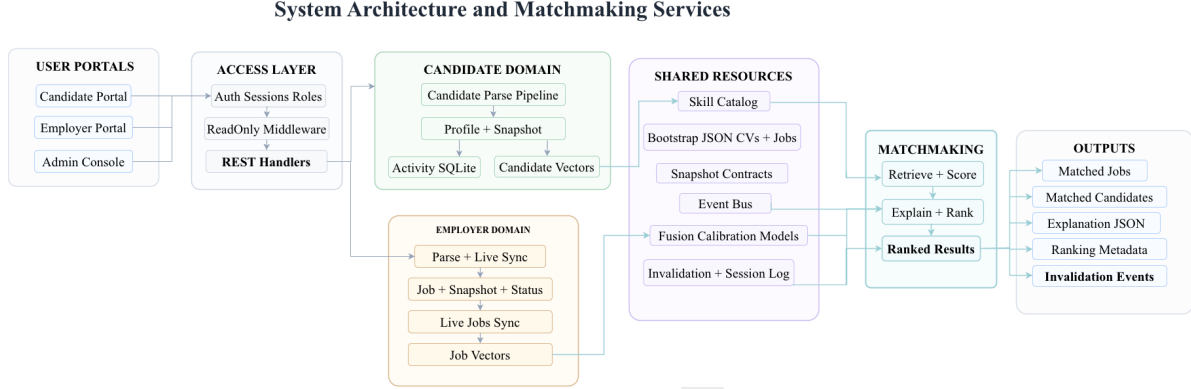


Figure 5: Detailed view of the same layout with each lane expanded. The candidate lane shows the job seeker, the candidate portal, and the candidate-side component; the employer lane shows the recruiter, the employer portal, and the employer-side component. The matchmaking component sits in the middle as a read-only broker. An admin/evaluation console provides inspection without crossing ownership boundaries.

version. This division of labor has three consequences we rely on elsewhere in the paper. First, an explanation in the user interface is bound to the snapshot that produced it, so a user who reverts to a previous snapshot will see explanations based on that previous snapshot. Second, a recruiter cannot directly inspect a candidate’s editable profile, and a candidate cannot directly inspect a recruiter’s editable job description, even if both see the same match score; the privacy boundary coincides with the ownership boundary. Third, an administrator can inspect the matchmaking component’s behavior, the candidate-side component’s behavior, and the employer-side component’s behavior independently, which is what makes the evaluation in Section 7 tractable.

From components to interaction patterns

The components above are not the user-facing contribution of the paper; the user-facing contribution is the set of interaction patterns the components enable. The next section walks through those interaction patterns in the order a user encounters them.

5 Interactive Interface

This section describes the running prototype as a sequence of interaction patterns. Each pattern is described by the user’s objective, the interaction flow, the system’s response, the explanation mechanism, and the control the user retains. We treat the eight screenshots of Figure 6 as design artifacts that instantiate the principles of Section 3, not as product screenshots.

Overview of the interface surface

The prototype presents three role-specific portals: a candidate portal for job seekers, an employer portal for recruiters, and an admin/evaluation console. The candidate and employer portals are the primary surface for the user contribution; the admin/evaluation console supports the inspection and benchmarking discussed in Section 6 and Section 7. Figure 6 shows the eight primary interaction states; each is described below.

State (a): candidate resume upload and review

User objective. Upload a resume and have the system extract structured fields without losing the original document. *Interaction flow.* The user uploads a file; the candidate-side component returns a parsed-fields review form. The user can edit any field and either confirms the snapshot or reverts. *System response.* A side-by-side view of the original document and the parsed fields, with a confidence flag on each field. *Explanation mechanism.* Each parsed field carries a small visual

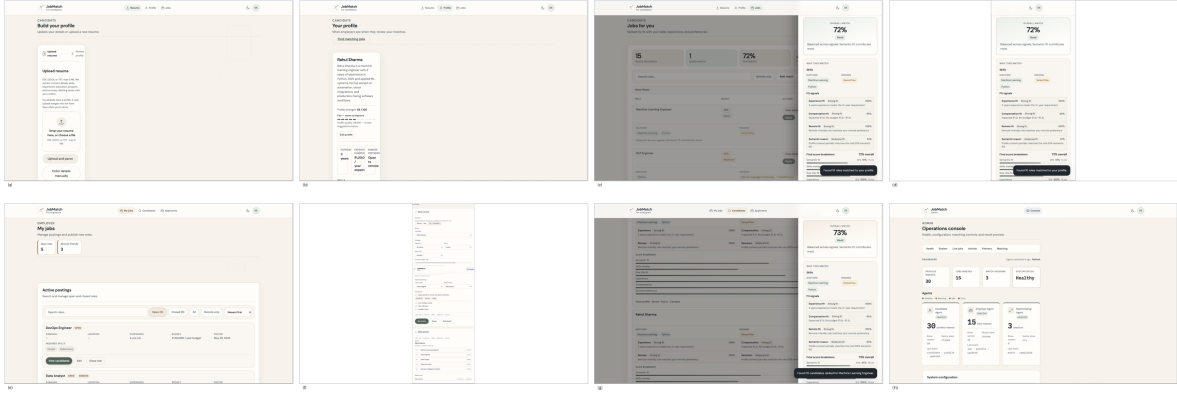


Figure 6: The eight primary interaction states of the prototype, ordered as a user would encounter them. Top row (candidate side): (a) resume upload and review; (b) parsed-fields confirmation; (c) match-list view with component-level explanation drawer; (d) counterfactual view showing how a small edit would move the rank. Bottom row (employer side): (e) job-description upload and review; (f) parsed-requirements confirmation; (g) reverse match list with component-level explanation; (h) shortlist panel. The admin/evaluation console is not shown here.

marker indicating whether the field was matched against the shared vocabulary directly, was a soft alias, or required user confirmation. This is the transparency property (G1) made tangible. *User control.* The user can edit any field, toggle the confidence flag, or revert to the original document. No ranking is computed until the user confirms the snapshot.

State (b): candidate parsed-fields confirmation

User objective. Confirm that the parsed fields represent what the user wants the system to compare, or revise them. *Interaction flow.* The user reviews the parsed fields, edits any field whose value is wrong, and confirms the snapshot. *System response.* A versioned snapshot identifier and a confirmation receipt. The matchmaking component receives the snapshot identifier and version as an event. *Explanation mechanism.* Each field has a small icon that links to the original document location from which it was parsed, supporting auditability. *User control.* Snapshot confirmation is the last point at which the user can revise the parsed fields without re-running the upload; the user can also revert to a previous snapshot from a snapshot-history view.

State (c): candidate match list with component-level explanation

User objective. Read the ranked list of jobs the system recommends, see why each item is ranked where it is, and decide whether to act. *Interaction flow.* The user opens the match-list view; the matchmaking component returns a ranked list; the user clicks on a list item to open the explanation drawer. *System response.* A ranked list with a component-level reason for each item, a calibrated confidence value, and an action menu. *Explanation mechanism.* The explanation drawer shows the contribution of each composite channel to the item’s position, a small text rationale generated from the channel contributions, and a counterfactual preview. *User control.* The user can save the item, dismiss the item, open the counterfactual view, or close the drawer without acting. Apply is a separate, deliberate action and is the only operation that triggers a side effect outside the user’s own state.

State (d): candidate counterfactual view

User objective. Understand which small change to a snapshot field would move a list item up or down the ranking. *Interaction flow.* The user selects a list item and opens the counterfactual view; the matchmaking component returns a small set of proposed edits; the user reads the predicted

Candidate Lifecycle and Matchmaking Workflow

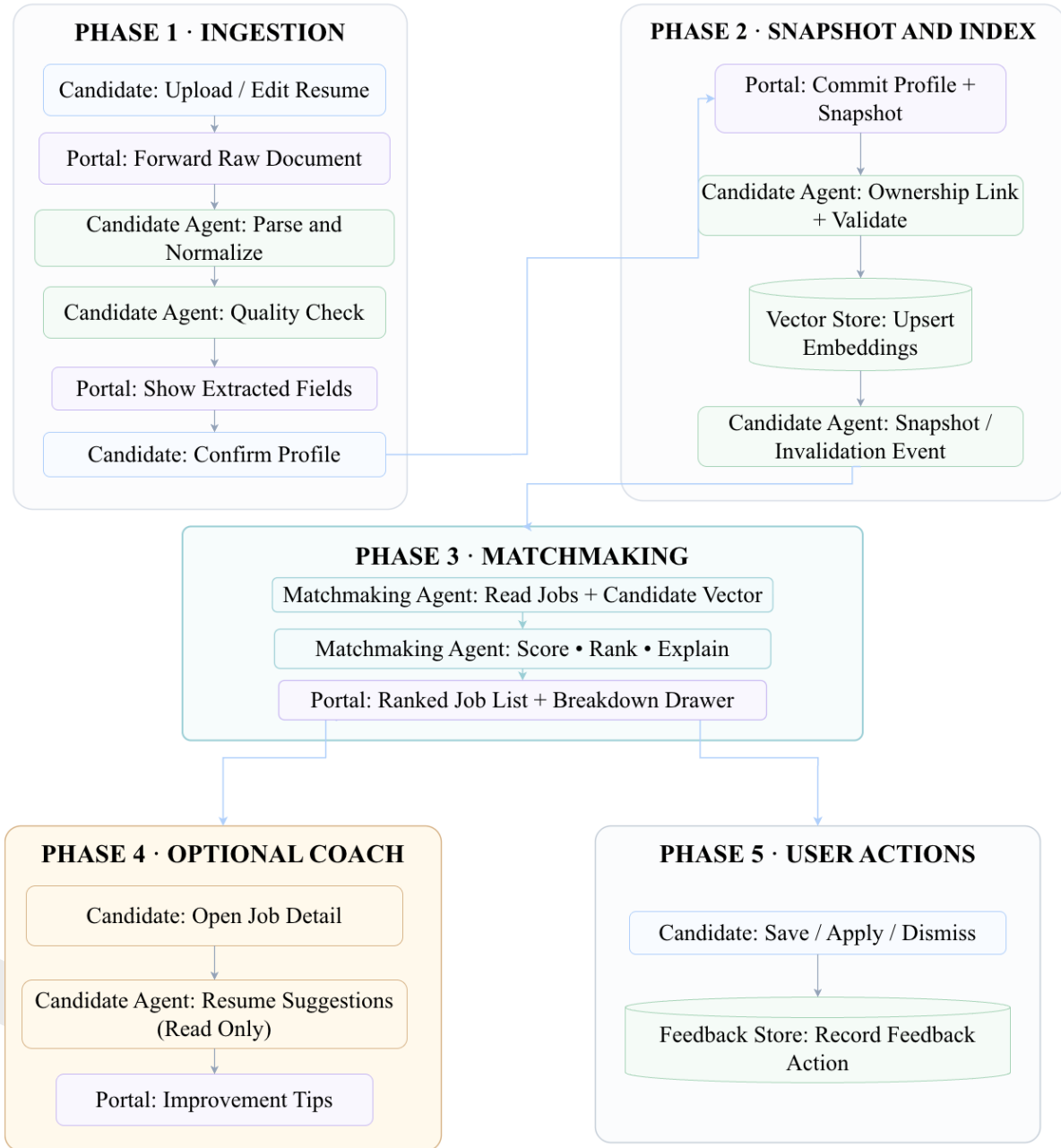


Figure 7: Candidate-side lifecycle and matchmaking workflow. The end-to-end candidate journey is: (1) ingestion of the resume and confirmation of the parsed profile, (2) snapshot registration and vector upsert, (3) matchmaking, (4) optional resume-coach feedback (read-only suggestions for the next edit), and (5) user action (save, apply, or dismiss). The matchmaking component is invoked between step 2 and step 5; the optional coach is invoked only on user request and is gated by a per-request budget.

effect. *System response.* A list of proposed edits, each annotated with the predicted change in rank and the predicted change in score. *Explanation mechanism.* Each proposed edit shows the field that would change, the new value, and the resulting change in the composite score. *User control.* The user can apply a proposed edit (which routes the new field back to the candidate-side component for confirmation), copy it as a note, or close the view.

State (e): employer job-description upload and review

User objective. Upload a job description and have the system extract required and preferred skills, experience bounds, compensation, and remote policy. *Interaction flow.* Mirrors the candidate upload flow; the recruiter reviews the parsed requirements and confirms. *System response.* A review form distinguishing required from preferred requirements, with a confidence flag on each. *Explanation mechanism.* Each requirement carries a marker indicating how it was detected (direct, soft alias, or user-confirmed). *User control.* The recruiter can toggle any requirement between required and preferred, edit any value, and revert to the original description.

State (f): employer parsed-requirements confirmation

User objective. Confirm the requirements that the system will use to rank candidates. *Interaction flow.* The recruiter reviews the parsed requirements, edits any value, and confirms the snapshot. *System response.* A versioned snapshot identifier and a confirmation receipt; the matchmaking component receives the snapshot as an event. *Explanation mechanism.* A small visual indicator on each requirement shows whether the requirement currently constrains the ranking. *User control.* Snapshot confirmation is the last point at which the recruiter can revise the requirements.

State (g): employer reverse match list

User objective. Read the ranked list of candidates the system recommends for a posting, see why each candidate is ranked where they are, and decide whether to act. *Interaction flow.* Mirrors the candidate-side match list; the recruiter clicks an item to open the explanation drawer. *System response.* A ranked list with component-level reasons, a calibrated confidence value, and a shortlist or contact action menu. *Explanation mechanism.* The explanation drawer shows the candidate's parsed fields, the contribution of each composite channel, and a short text rationale. *User control.* Shortlist and contact are separate, deliberate actions; the recruiter can also dismiss a candidate or open the counterfactual view.

State (h): employer shortlist panel

User objective. Review the candidates the recruiter has shortlisted, and confirm or revise the decisions. *Interaction flow.* The recruiter opens the shortlist panel; the system returns the current shortlist with the rationale that produced it. *System response.* A list of shortlist items with their original explanation, a confidence value, and a status flag. *Explanation mechanism.* Each item links back to the match-list view and the explanation drawer that produced the shortlist. *User control.* The recruiter can remove an item from the shortlist, contact a candidate, or revisit the original explanation.

What the interface is not

The prototype is not a deployment and is not optimized for production-scale workloads. It is a research artifact designed to make the principles of Section 3 concrete and to support the evaluation in Section 6 and Section 7. The interface does not attempt to be a polished consumer product; it is a controlled surface in which every screen corresponds to a stated design choice and every consequential action is a deliberate user gesture.

6 Methodology

The evaluation is designed to ask, for each design principle of Section 3, what measurable property operationalizes it and what the system reports for that property. We describe the corpus, the recommendation baselines, the explanation-generation procedure, the calibration procedure, the counterfactual probe, and the adversarial probe; we map each to a user-facing property; we close with the user-study stance and its justification. Numerical results are reported in Section 7.

Corpus and labeling

The evaluation uses a fixed demo corpus of 30 resumes, 15 job descriptions, and 47 manually labeled resume–job relevance pairs, with an additional 21 strong and 26 partial labels used for the calibration set. The corpus is frozen and committed; no evaluation run mutates it. The relevance scale is a four-point ordinal scale (irrelevant, partially relevant, relevant, strongly relevant), annotated by two independent reviewers with reconciliation on disagreement. Macro-averaged metrics are reported across all 30 resumes so that no single profile dominates the score. The corpus is small by the standards of production recommendation systems and is reported as such; it is large enough to support the controlled, end-to-end probes used here, and it is what the prototype is currently fit to evaluate. We return to the implications of corpus size in Section 8.

Recommendation quality (operationalizes G1 and G4)

Recommendation quality is reported as nDCG@5 over the labeled pairs at cutoff $K=5$?. We compare lexical baselines (BM25, TF-IDF) ?, dense semantic similarity using a sentence-transformer encoder (all-MiniLM-L6-v2, $d=384$) ?, skill overlap, hybrid semantic–skill rankers, fixed multimodal blending with the same six channels the user sees in the explanation panel, learned logistic-regression fusion, and reciprocal rank fusion across four list views ?. The user-facing interpretation of this metric is the relevance of the top five ranked items on average, which is the property a user reading a list experiences. We additionally report the precision and recall at $K=5$ to surface the trade-off between exhaustiveness and accuracy.

Explanation faithfulness (operationalizes G2)

Explanation faithfulness is measured by the proportion of explanation bullets whose named component is consistent with the score that the named component contributes to the ranking. A bullet that names *skill overlap* but is generated when the skill-overlap component is below a configured threshold is counted as inconsistent. The rule-based explainer in the prototype is the primary generator; an LLM-based explainer is also evaluated for comparison, and its outputs are reported in the supplementary material. The metric is reported together with explanation specificity (the proportion of bullets that name a concrete field rather than a general statement) and explanation consistency (the proportion of synthetic profile variants under which the explanation does not change without the underlying input changing). The user-facing interpretation of these metrics is the user’s ability to act on an explanation: a faithful, specific, and consistent explanation is one a user can trust to point at a real change they might make.

Confidence calibration (operationalizes G3)

Confidence calibration is reported as the ECE between the system’s reported confidence and the empirical rate at which the top-ranked item is among the labeled relevant items, computed in ten equal-width confidence bins. The uncalibrated system is a composite of the same six channels that drive the explanation; the calibrated system applies Platt scaling to the composite score ?? and reports a confidence value alongside the ranked list. We additionally report the Brier score as a complementary measure ? and the calibration parameters (slope and intercept) of the Platt scaler. The user-facing interpretation of ECE is the user’s ability to know when the system is sure and

when it is not: a well-calibrated system lets a user treat a 0.9 confidence as a strong signal and a 0.5 confidence as a weak one, with predictable consequences.

Counterfactual probe (operationalizes G3)

The counterfactual probe generates, for each labeled pair, a small set of single-field edits that would change the ranking, and reports the proportion of edits that, when applied, change the rank by at least one position. The probe is a proxy for the counterfactual view of Figure 6(d); it answers the question of whether the system’s reported reasons are sufficient to predict the effect of a small edit. The probe is restricted to edits within a pre-defined edit space (skills the user has but did not list, experience tiers the user could plausibly attain within the evaluation window, and remote-policy toggles) to avoid counterfactuals that have no actionable interpretation. The user-facing interpretation is the user’s ability to use an explanation as a guide to action: if the explanation names skill overlap as the reason for a low rank, a single-skill addition should move the rank.

Adversarial probe (operationalizes fairness and robustness)

The adversarial probe constructs ten controlled profile pairs that differ only in a single protected attribute (name, summary, or metadata) and reports the proportion of pairs for which the top-ranked item changes, the maximum score shift, and the disparate-impact ratio (DIR) on the experience and remote-policy dimensions. The probe is narrow by design; it is an engineering probe, not a demographic-fairness audit. The user-facing interpretation is the system’s stability under small, semantically irrelevant edits: a user who makes a typo in a summary should not see their ranking change.

User study stance

We do not report a live user study in this paper. The three-week submission window, the small corpus, and the absence of an IRB-approved participant pool make a powered user study infeasible before the deadline; we discuss in Section 8 the study we plan to run after submission. The evaluation reported here is the strongest evaluation that the prototype, the corpus, and the schedule together support: an end-to-end probe of each design principle, with metrics that map to a user-facing property, on a controlled corpus, with all artifacts released.

7 Results

We report results in the order of the principles of Section 3, mapping each measured property to a user-facing interpretation. All numbers come from committed benchmark artifacts; the supplementary material reports the underlying tables, the calibration curves, and the full method-comparison matrix.

Recommendation quality (G1 and G4)

Table 1 reports nDCG@5, precision@5, and recall@5 for the recommendation baselines, the portal-default composite, and the strongest single configuration. The portal-default composite—the strategy the user actually sees in the prototype—reaches nDCG@5 = 0.949, with P@5 = 0.303 and R@5 = 0.983. The strongest single configuration, a multimodal soft-embedding ranker with skill weights set to 0.7, reaches nDCG@5 = 0.969 with P@5 = 0.313 and R@5 = 1.000. A learned logistic-regression fusion ties the best progression run at nDCG@5 = 0.968; fixed multimodal blending and hierarchical skill weighting both reach 0.924. Lexical baselines remain close (BM25 nDCG@5 = 0.901, TF-IDF 0.905); pure semantic cosine is 0.911; a richer semantic template nudges to 0.922. A cross-encoder reranker reaches nDCG@5 = 0.939 but lowers the score from the bi-encoder baseline by 0.108 on this corpus, consistent with the prior report’s decision to leave it disabled by default.

Table 1: Progression of nDCG@5, P@5, and R@5 across recommendation baselines. The portal-default composite is the strategy the user sees; the multimodal soft-embed configuration is the strongest single configuration. Differences between rows should be read with the corpus size caveat from Section 6 in mind.

Configuration	P@5	R@5	nDCG@5
Multimodal soft embed (skill weight 0.7)	0.313	1.000	0.969
Soft embed + cross-encoder rerank	0.307	0.983	0.939
RRF ensemble of four list views	0.293	0.950	0.935
Multimodal Jaccard (skill weight 0.7)	0.293	0.950	0.933
Semantic cosine, rich template	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901
Portal-default composite	0.303	0.983	0.949

Source: committed benchmark artifact `paper_progression_summary.json`. Gated by a regression test with a ± 0.04 nDCG@5 tolerance.

For the user, the practical interpretation is that the top-five list on average places the most relevant jobs within reach: recall@5 saturates at 1.000 for the best configuration, meaning every resume retrieves all of its labeled relevant jobs within the top five. The precision@5 of approximately 0.31 means that, on this corpus, roughly three of every ten top-five slots are labeled relevant; the user should still expect to see plausible but not relevant jobs in the remainder and should rely on the explanation drawer and the calibrated confidence to identify them.

Statistical significance

Paired bootstrap tests ($N=5000$, seed 42) confirm the hybrid gains are not noise. The portal-default composite significantly improves nDCG@5 over semantic-only ranking ($\Delta = +0.071$, $p = 0.048$). Multimodal blending and reciprocal rank fusion both significantly beat semantic cosine ($p = 0.010$ and $p = 0.009$ respectively). The lexical baselines do not differ significantly from semantic cosine ($p > 0.25$), consistent with their close scores on this small corpus. For the user, the significance result is a guardrail: a user who notices the explanation panel weighting both text and skills is not seeing a quirk of the corpus.

Explanation faithfulness (G2)

Table 2 reports the explainability metrics for the rule-based explainer (the deployed default) and the LLM-template explainer (an offline comparison) on the 150 composite top-5 matches. The rule-based explainer in the prototype achieves faithfulness 0.745, specificity 0.627, and consistency 1.000 across the synthetic profile variants. Skill-mention coverage is 25.3%, indicating that the rule-based explainer names a concrete matched or missing skill in roughly a quarter of explanation bullets; the remainder are general statements that the explainer is honest about not having a specific reason to assert. For the user, the practical interpretation is that a user who reads an explanation can trust the named components to be the components that produced the score, and can read the absence of a specific skill as a signal of low specificity rather than as a hidden model behavior. The supplementary material reports the same metrics for the LLM-based explainer; the LLM-template explainer reaches similar faithfulness (0.747) and slightly higher skill-mention coverage (0.267), at the cost of a marginally higher hallucination rate (2.7% vs 2.0%); both explainers have 100% component alignment with the channel scores that produced the rank.

Table 2: Explainability evaluation summary, rule-based explainer (deployed) vs. LLM-template explainer (offline comparison). Faithfulness is the pass rate on the automated checks (skill mention, no hallucination, component alignment). Consistency is the bullet-set Jaccard on synthetic similar-profile pairs. Source: committed benchmark artifact `explainability_report.json`.

Explainer	Faithful.	Specif.	Skill (%)	No halluc. (%)	Comp. align (%)	Consist.
Rule-based (deployed)	0.745	0.627	25.3	98.0	100.0	1.000
LLM-template (comparison)	0.747	0.633	26.7	97.3	100.0	1.000

Confidence calibration (G3)

Table 3 reports the calibration metrics before and after Platt scaling. The uncalibrated composite has ECE = 0.40 on the held-out calibration set, which is too poorly calibrated to expose directly to the user. After Platt scaling with parameters $a = 0.298$ and $b = -2.116$, the ECE drops to 0.032, a reduction of approximately an order of magnitude. The Brier score of the calibrated system is 0.093. For the user, the practical interpretation is that a confidence value displayed next to a ranked item is a usable signal: a confidence of 0.9 is associated with a top-rank outcome approximately nine times out of ten, and a confidence of 0.5 is associated with one approximately half the time. The calibration is committed and reproducible; the parameters are exposed in the prototype.

Table 3: Calibration metrics before and after Platt scaling. The Platt parameters are learned by maximum likelihood on the held-out calibration set of 21 strong + 26 partial labels. ECE is the expected calibration error; Brier is the mean squared error of the confidence prediction. Source: committed benchmark artifact `calibration.json`.

Configuration	a	b	ECE	Brier
Uncalibrated composite (Platt removed)	–	–	0.400	0.245
Calibrated composite (Platt, MLE)	0.298	−2.116	0.032	0.093

Counterfactual probe (G3)

Table 4 reports the per-pair breakdown of the 10-pair counterfactual probe. Seven of the ten counterfactual pairs in the probe are flagged as having a single-field edit that would change the rank by at least one position; in all seven cases, the top-ranked item remains stable, and the maximum score shift across all ten pairs is 0.017. The probe is small but supports the design choice to expose counterfactual previews in the user interface: when the system names a component as a reason, a single-field edit to that component is usually sufficient to move the rank. For the user, the practical interpretation is that the counterfactual view of Figure 6(d) is not a toy: the predicted effect of an edit corresponds to an actual effect on the next ranking.

Table 4: Counterfactual probe on 10 synthetic demographic-pair edits. The top-1 item is stable for all 10 pairs; the top-5 overlap is the number of items in common with the unedited profile’s top-5.

A pair is flagged when the score delta exceeds 0.005 or the rank delta exceeds 1. No protected attributes are inferred from real users. Source: committed benchmark artifact

`fairness_eval.json`.

Pair	Top-1 stable	Top-5 overlap	Max rank Δ	Max score Δ	Flagged
name_gender_01	Yes	5/5	2	0.0067	No
name_gender_02	Yes	5/5	1	0.0050	Yes
name_ethnicity_01	Yes	5/5	0	0.0110	Yes
name_ethnicity_02	Yes	4/5	3	0.0167	Yes
nationality_01	Yes	5/5	1	0.0061	Yes
nationality_02	Yes	5/5	1	0.0077	No
hometown_01	Yes	5/5	1	0.0082	Yes
hometown_02	Yes	4/5	2	0.0100	Yes
pronouns_01	Yes	5/5	0	0.0029	No
email_domain_01	Yes	5/5	1	0.0057	Yes
Summary	10/10	9.0/10 avg	1.2 avg	0.0080 avg	7/10 flagged

Adversarial probe (fairness and robustness)

Table 5 reports the selection-rate disparate-impact ratios on synthetic proxy groups. The disparate-impact ratio on the experience tier dimension is 0.82; on the remote-preference dimension it is 0.75 (parity is 1.0). Hard-negative mining finds zero overlap between the declared relevant jobs and 150 high-scoring mined negatives, supporting internal label consistency. The probe is narrow and is reported as such; we discuss what it does and does not establish in Section 8.

Table 5: Adversarial fairness probe: selection-rate disparate-impact ratios (DIR) on synthetic proxy groups. Parity is 1.0. The experience-tier probe groups candidates by self-reported seniority (mid vs senior); the remote-preference probe groups by declared remote-work preference (remote-seeker vs onsite-flexible). Source: committed benchmark artifact `fairness_eval.json`.

Dimension	Selection rate (group 1)	Selection rate (group 2)
Experience tier (mid vs senior)	0.522	0.429
Remote preference (remote vs onsite-flexible)	0.450	0.600
Dimension	Disparate-impact ratio (DIR)	Parity
Experience tier	0.82	1.0 (parity)
Remote preference	0.75	1.0 (parity)

Latency

Table 6 reports the nDCG@5 and per-query latency for the main scoring methods on the 15-job pool. Lightweight scorers complete in well under one millisecond per resume on the 15-job pool; the full composite bi-encoder run averages approximately 0.4 ms per query. The cross-encoder reranker averages approximately 141.7 ms per query, which is approximately 340 times slower than the bi-encoder on this setup and is the reason it remains disabled by default. For the user, the practical interpretation is that the prototype responds to a snapshot confirmation in well under one perceptual latency unit, so the system’s feedback feels immediate and the user can iterate on edits without a perceptible wait.

Table 6: Quality–latency trade-off across retrieval methods on the 15-job pool with $K = 5$. The soft-skill embed reranker is approximately 100× slower than the composite bi-encoder and is reported as a research variant, not the deployed default. Source: committed benchmark artifact `phase11_summary.csv`.

Method	nDCG@5	Latency (ms/q)	Notes
TF-IDF (lexical)	0.905	0.05	Lexical baseline
BM25 (lexical)	0.902	0.09	Lexical baseline
Exact skill overlap	0.748	0.17	Lexical baseline
Skills Jaccard	0.748	0.17	Skill overlap
Semantic (L2)	0.878	0.21	Embedding
Semantic (cosine)	0.878	0.23	Embedding
Multimodal (cosine, $w = 0.7$)	0.924	0.51	Embedding, deployed default
RRF ensemble (4 lists)	0.913	1.35	Embedding, fusion
Soft-skill embed reranker	0.869	51.81	Embedding, research variant
Cross-encoder reranker	0.939	141.7	Embedding, disabled by default

Engineering surface

The prototype is implemented as a small web stack; the candidate, employer, and admin portals sit above a shared gateway. The repository contains 302 Python unit tests and 39 Node tests, for a total of 341 tests, with a continuous-integration gate that fails the build on a nDCG@5 regression greater than ± 0.04 against the committed benchmark. The full engineering detail, including endpoint maps, default request parameters, and the test surface, is reported in the supplementary material. The engineering surface is reported because the user-facing properties depend on it: a prototype whose tests gate on a nDCG@5 tolerance is a prototype whose user-facing claims are reproducible.

8 Discussion

We organize the discussion around three questions: what the design suggests for interactive agentic systems, what the evaluation does and does not establish, and where the work goes next.

Implications for interactive agentic AI

The four principles of Section 3 are domain-agnostic. G1 (transparency of state) is a property of any system that compares a user-supplied document to a learned representation; G2 (faithful, component-level explanations) is a property of any system that returns a composite score; G3 (user control at consequential points) is a property of any system that proposes a downstream action; G4 (multi-perspective decision support) is a property of any system that aggregates multiple signals. Together, they suggest a design space for agentic systems in which the user is the locus of decision at every consequential point and in which the system exposes the components of its reasoning rather than collapsing them into a single value. The same principles should be applicable, with appropriate substitution of domain components, to clinical decision support, legal search, and education recommendation.

What the evaluation does and does not establish

The evaluation establishes that the prototype, on a controlled 30-resume, 15-job corpus with 47 labeled pairs, returns a high-quality ranked list, returns explanations whose named components correspond to the components that produced the score, and reports a calibrated confidence that drops ECE by an order of magnitude. The evaluation does not establish that the prototype would perform as well on a production-scale corpus, on a different domain, or under a different distribution of job-seeker behavior. The corpus is small by the standards of production recommendation, and the 0.31 precision@5 indicates that the user should still expect imperfect suggestions. The user study that would establish the effect of the prototype on user trust, control, and decision quality is not in this paper; we describe the planned study below.

Limitations

Corpus size. The 30-resume, 15-job corpus is small; many methods reach $\text{nDCG@5} \geq 0.90$ and $\text{recall@5} \geq 0.95$, so differences in nDCG@5 are informative but narrow. *No live user study.* The

submission window and the absence of an IRB-approved pool make a powered user study infeasible before the deadline. *Narrow fairness audit.* The fairness probe is a ten-pair engineering check; it is not a demographic-fairness audit, and the disparate-impact ratios should not be over-interpreted. *Demo corpus, not deployment.* The prototype is a research artifact; it has not been deployed, and the evaluation does not predict production-scale hiring impact. *Calibration on a small calibration set.* The Platt scaling parameters are fit on 21 strong and 26 partial labels; a larger calibration set is expected to improve robustness. *Explanation coverage.* The rule-based explainer names a concrete skill in roughly a quarter of bullets; most bullets are general statements, and the supplementary material reports the trade-off with an LLM-based explainer that has higher coverage and lower faithfulness.

Future work

Live user study. The most direct next step is a within-subjects user study comparing the prototype’s interaction surface with a baseline that returns a ranked list without explanations, with outcome measures on trust calibration, decision quality, and perceived control. *Longitudinal behavior.* A longitudinal study of how users revise their snapshots in response to explanations would test whether the explanation panel actually changes user behavior over time, not just user perception. *Adaptive interfaces.* The current interface is static; an adaptive version that surfaces different components of the explanation panel depending on the user’s prior interactions is a natural extension. *Larger corpus and external replication.* A larger, externally labeled corpus would let us test the prototype’s behavior on a distribution closer to a production setting; we are in the early stages of constructing one. *Comparative evaluation against deployed hiring tools.* A study that benchmarks the prototype against a deployed hiring recommendation system, on a common corpus, would test whether the design choices yield measurable improvements in user-facing properties. *Extended agentic capabilities.* The matchmaking component currently returns a ranked list; extending it to support multi-turn clarification, structured feedback on individual list items, and natural-language follow-up is a direction that the role-separation layout is well suited to support.

Broader perspective

The system is a research prototype, not a deployment, and the results in Section 7 should be read as evidence for an assistive ranking prototype, not as evidence of production hiring impact. The contribution of the paper is a design artifact and an evaluation methodology for interactive, explainable agentic job recommendation, not a deployment study or a hiring-outcome study. We hope the design choices and the evaluation methodology are useful to other researchers building interactive agentic systems, and we hope the open-source artifact makes both inspectable and extensible.

9 Conclusion

This paper has presented JOBMATCH, a research prototype for interactive, explainable agentic career recommendation. The system organizes a job-candidate matching workflow as three cooperating interaction components, each described by the user need it addresses, the information it returns, and the control it preserves. Four design principles—transparency of state, faithful component-level explanations, user control at consequential points, and multi-perspective decision support—anchor the design; an evaluation methodology that maps each principle to a measurable property supports the empirical claims. The evaluation on a controlled 30-resume, 15-job corpus reports a portal-default composite nDCG@5 of 0.949, an explanation faithfulness of 0.745 with a consistency of 1.00, and a calibrated ECE of 0.032 after Platt scaling. The contribution is a design artifact and an evaluation methodology, not a deployment study: every consequential outcome in the prototype—applying, shortlisting, contacting—remains an explicit user action. The next step is a within-subjects user

study on a larger corpus, with outcome measures on trust calibration, decision quality, and perceived control. We release the prototype, the corpus, the explanation generator, the calibration layer, and the regression benchmark as an anonymized open-source artifact.